SRE Platform Engineer — Interview Questions & Answers (Part 2)

Role: Site Reliability Engineer / Platform Engineering | **Level:** 7-10 Years

Section 3: Observability — Metrics, Logs, Traces (Q16–Q21)

Q16. What are the three pillars of observability and how do they relate?

Answer:

Pillar	What	Tool Examples
Metrics	Numeric measurements over time	Prometheus, CloudWatch, Datadog
Logs	Discrete event records	ELK, Loki, CloudWatch Logs
Traces	Request journey across services	Jaeger, X-Ray, Tempo

How they relate:

- **Alert on metrics** (error rate > 1%) → tells you SOMETHING is wrong
- **Search logs** to find WHAT is wrong (specific error messages)
- **Trace** to find WHERE in the call chain the error occurs

Example workflow:

1. Metric alert: p99 latency > 2s
2. Dashboard shows: `order-service` latency spike
3. Trace shows: `order-service → inventory-service` call takes 1.8s
4. Logs from `inventory-service`: "Connection pool exhausted"
5. Root cause: DB connection leak after recent deployment

Golden signals I always monitor (Google's 4):

- **Latency** — Response time (p50, p90, p99)
- **Traffic** — Requests per second
- **Errors** — Error rate (5xx / total)
- **Saturation** — Resource utilization (CPU, memory, connections)

Q17. How do you set up Prometheus monitoring for Kubernetes?

Answer:

Architecture:

```
App pods (expose /metrics) → Prometheus (scrape + store)
  → Grafana (dashboards)
  → Alertmanager (routing + notification → PagerDuty/Slack)
```

Deployment: Use **kube-prometheus-stack** Helm chart (Prometheus Operator + Grafana + Alertmanager + node-exporter + kube-state-metrics).

Key metric sources:

Source	What it monitors
node-exporter	CPU, memory, disk, network per node
kube-state-metrics	Pod status, replica counts, resource requests
cAdvisor	Container CPU, memory, I/O
App /metrics	Custom business metrics (orders/sec, queue depth)

ServiceMonitor for auto-discovery:

```yaml
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: my-app
spec:
  selector:
    matchLabels:
      app: my-app
  endpoints:
    - port: http
      path: /metrics
      interval: 15s
```

Recording rules for expensive queries:

```
  - record: job:http_requests:rate5m
    expr: sum(rate(http_requests_total[5m])) by (job)
```

Pre-compute and store the result instead of querying raw metrics every time.

Q18. How do you implement centralized logging in Kubernetes?

Answer:

Pattern: DaemonSet log collector → aggregator → storage

```
Container stdout/stderr → Node log file
  → Fluent Bit (DaemonSet on every node)
  → Elasticsearch / OpenSearch / Loki
  → Kibana / Grafana (query & dashboards)
```

Why Fluent Bit over Fluentd:

- 10x less memory (~15MB vs ~150MB)
- Written in C (faster than Ruby-based Fluentd)
- Sufficient for log forwarding; use Fluentd only if you need complex transformation

Structured logging standard I enforce:

```
{
  "timestamp": "2026-05-07T10:30:00Z",
  "level": "ERROR",
  "service": "order-service",
  "trace_id": "abc123",
  "message": "Payment gateway timeout",
  "duration_ms": 5002,
  "order_id": "ORD-9876"
}
```

Key rules:

- JSON format (not plaintext — enables querying)
- Include `trace_id` in every log (correlate with traces)

- Log at the right level: ERROR (actionable), WARN (investigate), INFO (audit), DEBUG (dev only)
- Set retention policies: 7 days hot, 30 days warm, 90 days cold (S3/Glacier)

Q19. Explain distributed tracing. How does it work under the hood?

Answer:

Concept: A trace follows a single request across multiple microservices. Each service creates a **span** (a unit of work with start/end time). Spans are linked by a shared **trace ID**.

```
User Request (trace_id: abc123)
  └─ API Gateway (span 1: 200ms)
       └─ Order Service (span 2: 150ms)
            ├─ Inventory Service (span 3: 80ms)
            └─ Payment Service (span 4: 60ms)
                 └─ Stripe API (span 5: 45ms)
```

How it works:

1. First service generates a `trace_id` and `span_id`
2. Passes both via HTTP header (`traceparent: 00-abc123-span1-01`)
3. Each downstream service creates a new span with the same trace_id
4. All spans are sent to collector (Jaeger/Tempo/X-Ray)
5. Collector stitches spans into a trace tree

W3C Trace Context is the standard header format. OpenTelemetry is the standard SDK.

Sampling strategies:

- **Head-based:** Decide at the start whether to trace (1% of requests). Simple but misses rare errors.
- **Tail-based:** Decide after completion. Keep all traces with errors or high latency. More expensive but catches important traces.

Q20. How do you build effective Grafana dashboards?

Answer:

Dashboard hierarchy (top-down):

1. **Executive dashboard:** 3-4 panels. Availability SLO (current %), error budget remaining, active incidents.
2. **Service overview:** Per-service golden signals (latency, traffic, errors, saturation).
3. **Deep-dive:** Per-service detailed metrics — CPU, memory, pod count, queue depth, DB connections.

Design rules:

- **RED method for services:** Rate, Errors, Duration
- **USE method for infrastructure:** Utilization, Saturation, Errors
- Time range selector at the top
- Annotations for deployments (vertical line showing when v2.3 was deployed)
- Variable dropdowns for namespace, service, environment

Anti-patterns:

- ❌ 50-panel dashboard (nobody reads it)
- ❌ Dashboard without clear ownership
- ❌ Metrics without context (what's the threshold? what's normal?)

Q21. How do you implement alerting that doesn't cause alert fatigue?

Answer:

Rules:

1. **Every alert must have a runbook** — Link in the alert annotation
2. **Alert on symptoms, not causes** — "Users seeing 5xx" not "CPU is 90%"
3. **Use multi-window burn rate** — (See Q7)

4. **Group related alerts** — Alertmanager `group_by: [cluster, namespace, service]`
5. **Route by severity:**
    - Critical → PagerDuty (page immediately)
    - Warning → Slack channel (investigate in business hours)
    - Info → Dashboard only

Alertmanager routing example:

```
route:
  group_by: [alertname, cluster]
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 4h
  receiver: slack-default
  routes:
    - match:
        severity: critical
      receiver: pagerduty-oncall
    - match:
        severity: warning
      receiver: slack-sre
```

Review cadence: Monthly alert review. For each alert: Was it actionable? Did someone act? If not → delete or downgrade.

Section 4: CI/CD & Infrastructure as Code (Q22–Q27)

Q22. How do you structure a production CI/CD pipeline?

Answer:

```
PR Created → Lint → Unit Test → SAST → Build → Image Scan → Push
PR Merged  → Deploy Dev → Integration Test → Deploy Staging → Smoke Test
             → Manual Approval → Deploy Prod (Canary) → Monitor → Promote
```

Key practices:

- **Immutable artifacts:** Build once, deploy the same artifact to all environments
- **Git SHA as image tag:** Traceable, immutable
- **Feature flags:** Decouple deployment from release
- **Canary deployment:** Route 5% traffic to new version, monitor error rate for 10 min
- **Automated rollback:** If canary error rate > baseline + 1%, auto-rollback
- **GitOps for K8s:** ArgoCD/Flux watches Git repo, syncs cluster state

---

Q23. How do you structure Terraform for multi-environment infrastructure?

Answer:

```
terraform/
├── modules/          # Reusable components
│   ├── vpc/
│   ├── eks/
│   ├── rds/
│   └── monitoring/
├── environments/
│   ├── dev/          # terraform.tfvars for dev
│   │   ├── main.tf   # Calls modules with dev params
│   │   └── terraform.tfvars
│   ├── staging/
│   └── prod/
└── shared/           # Cross-env resources (DNS, IAM)
```

Best practices:

- **Remote state** in S3 + DynamoDB locking
- **Separate state per environment** (blast radius isolation)
- **Module versioning** via Git tags (`source = "git::...?ref=v1.2.0"`)
- **`terraform plan` in PR** — Post plan as PR comment
- **`terraform apply` only on main merge** with environment approval gates
- **Drift detection** — Scheduled `terraform plan` to detect manual changes
- **Checkov/tfsec** in CI — Security scanning before apply

Q24. Explain GitOps. How does ArgoCD work?

Answer:

GitOps principle: Git is the single source of truth for infrastructure and application state. Changes happen via Git commits (PR → review → merge), not kubectl or console.

ArgoCD flow:

1. Developer updates K8s manifest in Git (via PR)
2. ArgoCD watches the Git repo (polls every 3 min or webhook)
3. Detects diff between Git (desired state) and cluster (actual state)
4. Auto-syncs or waits for manual approval (configurable)
5. Applies changes to cluster
6. Self-heals: if someone manually changes the cluster, ArgoCD reverts it

Key ArgoCD features:

- **App of Apps pattern:** One ArgoCD Application manages multiple Application CRs
- **ApplicationSets:** Template-based generation (one template → deploy to 10 clusters)
- **Sync waves:** Control deployment order (namespace first, then configmap, then deployment)
- **Diff visualization:** See exactly what will change before syncing

---

Q25. How do you manage Helm chart upgrades safely?

Answer:

```
# 1. Always diff before upgrading
helm diff upgrade my-release my-chart --values values.yaml

# 2. Use --atomic for auto-rollback on failure
helm upgrade --install my-release my-chart \
  --namespace production \
  --values values.yaml \
  --atomic \                # Rollback if any resource fails
```

```
  --timeout 5m \              # Fail if not ready in 5 min
  --wait                      # Wait for all pods to be ready

# 3. Pin chart versions
helm upgrade --install my-release my-chart --version 2.3.1
```

Best practices:

- **Never `helm upgrade` without `--atomic`** in production
- **Pin chart versions** — never use `latest`
- **Values files per environment** — `values-dev.yaml`, `values-prod.yaml`
- **Helm secrets** — Use `helm-secrets` plugin for encrypted values
- **Chart testing:** `helm template` → `kubeval` → `conftest` (OPA) in CI

Q26. How do you handle database migrations in CI/CD?

Answer:

Strategy: Separate migration from deployment

1. **Backward-compatible migrations only:**
   - ✅ Add column (nullable or with default)
   - ✅ Add index
   - ❌ Rename column (breaks old code)
   - ❌ Drop column (breaks old code)
2. **Two-phase approach for breaking changes:**
   - Phase 1: Add new column, deploy code that writes to both old + new
   - Phase 2: Backfill data, deploy code that reads from new
   - Phase 3: Drop old column (after all old code is gone)
3. **Migration execution:**
   - K8s **init container** or **Job** runs migrations before app starts
   - Or: Separate migration pipeline triggered before deployment
   - Use tools like Flyway (Java), Alembic (Python), or golang-migrate

4. **Safety checks:**
    - Lock-aware migrations (use `pt-online-schema-change` for MySQL)
    - Preview migration in staging with production-sized data
    - Automatic rollback script for every migration

Q27. How would you implement a self-service developer platform?

Answer:

Internal Developer Platform (IDP) components:

Component	Tool	Purpose
Service catalog	Backstage	Register, discover, and document services
Golden paths	Backstage templates	Scaffold new services with CI/CD, monitoring, K8s manifests
Self-service infra	Crossplane / Terraform + GitOps	Developers request infra via PR (DB, cache, queue)
Environment provisioning	ArgoCD ApplicationSets	PR-based preview environments
Secret management	External Secrets Operator	Developers reference secrets, SRE manages rotation
Observability	Grafana + Prometheus + Loki	Pre-built dashboards per service template

Philosophy: SRE builds the platform, developers use it. Developers shouldn't need to know Terraform, Helm, or kubectl to deploy a service.

Section 5: Linux, Scripting & Troubleshooting (Q28–Q32)

Q28. A server's load average is high but CPU usage looks normal. What do you check?

Answer:

Load average includes processes in **uninterruptible sleep (D state)** — usually waiting on I/O.

```
# 1. Check load vs CPU count
uptime
nproc  # If load > nproc, overloaded

# 2. Check for I/O wait
top  # Look at %wa (I/O wait)
iostat -x 1 5  # Check disk utilization, await time

# 3. Find processes in D state (waiting on I/O)
ps aux | awk '$8 ~ /D/'

# 4. Check disk I/O
iotop -o  # Show only processes doing I/O

# 5. If disk is the bottleneck:
# - Is it swap? (free -h, check if swap is active)
# - Is it a log file growing? (lsof +D /var/log)
# - Is it NFS mount hanging? (mount | grep nfs)
```

Common causes: Swap thrashing, slow NFS mount, saturated EBS volume (check IOPS limit), processes waiting on disk I/O.

Q29. Write a script to find the top 5 pods consuming the most memory in a cluster.

Answer:

```
#!/bin/bash
# Top 5 memory-consuming pods across all namespaces
kubectl top pods --all-namespaces --sort-by=memory | head -7
# (head -7 = header + 5 results + buffer)

# More detailed with percentage of limit:
kubectl get pods --all-namespaces -o json | \
  jq -r '.items[] |
```

```
    select(.status.phase=="Running") |
    .metadata.namespace + "/" + .metadata.name + " " +
    (.spec.containers[0].resources.limits.memory // "no-limit")' | \
  sort | head -20
```

Python version for automation:

```python
#!/usr/bin/env python3
import subprocess, json

result = subprocess.run(
    ["kubectl", "top", "pods", "--all-namespaces", "--no-headers"],
    capture_output=True, text=True
)
pods = []
for line in result.stdout.strip().split("\n"):
    parts = line.split()
    ns, name, cpu, mem = parts[0], parts[1], parts[2], parts[3]
    mem_mi = int(mem.replace("Mi", ""))
    pods.append((mem_mi, ns, name))

pods.sort(reverse=True)
print(f"{'Memory':>10} {'Namespace':<20} {'Pod'}")
for mem, ns, name in pods[:5]:
    print(f"{mem:>8}Mi {ns:<20} {name}")
```

Q30. How do you troubleshoot network connectivity between pods?

Answer:

```
# 1. Test basic DNS resolution
kubectl exec -it <pod> -- nslookup <service-name>
kubectl exec -it <pod> -- nslookup <service-name>.
<namespace>.svc.cluster.local

# 2. Test TCP connectivity
kubectl exec -it <pod> -- nc -zv <target-service> <port>
# or: curl -v http://<service>:<port>/health

# 3. Check NetworkPolicies
kubectl get networkpolicies -n <namespace>
# If default-deny exists, check if allow rules cover this path

# 4. Check if service has endpoints
kubectl get endpoints <service-name> -n <namespace>
```

```
# Empty endpoints = no matching pods (label mismatch or pods not ready)

# 5. Check kube-proxy / iptables
kubectl logs -n kube-system -l k8s-app=kube-proxy

# 6. Temporary debug pod in same namespace
kubectl run netshoot --image=nicolaka/netshoot -it --rm -- /bin/bash
# Then: curl, dig, tcpdump, iperf, etc.
```

Most common issues:

1. NetworkPolicy blocking traffic
2. Service selector doesn't match pod labels
3. Pod not ready (failing readiness probe) → not in endpoints
4. DNS resolution failure (CoreDNS pods unhealthy)

Q31. Explain how you'd automate SSL certificate renewal.

Answer:

In Kubernetes: Use **cert-manager** + Let's Encrypt:

```
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: letsencrypt-prod
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
    privateKeySecretRef:
      name: letsencrypt-account-key
    solvers:
      - dns01:
          route53:
            region: us-east-1
---
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: app-tls
spec:
  secretName: app-tls-secret
  issuerRef:
    name: letsencrypt-prod
    kind: ClusterIssuer
  dnsNames:
```

```
    - app.company.com
    - "*.company.com"
```

cert-manager automatically renews 30 days before expiry. Zero manual intervention.

On AWS: ACM (AWS Certificate Manager) certificates auto-renew for free. No scripting needed.

Monitoring: Alert if certificate expires within 14 days (in case auto-renewal fails):

```
- alert: CertificateExpiringSoon
  expr: certmanager_certificate_expiration_timestamp_seconds - time() < 14*24*3600
  labels:
    severity: warning
```

Q32. How do you perform capacity planning for a growing platform?

Answer:

Data-driven approach:

1. **Baseline current usage:**
   - Peak CPU/memory across all nodes (last 30 days)
   - Request rate trends (Prometheus: `rate(http_requests_total[7d])`)
   - Storage growth rate (GB/month)
2. **Project forward:**
   - Business input: "We expect 2x users in 6 months"
   - Technical: "Each 1000 users = ~2 pods = ~1 vCPU + 2GB RAM"
   - Add 30% buffer for spikes
3. **Key metrics to track:**
   - Node utilization > 70% sustained → add capacity
   - Pod pending time > 30s → nodes too slow to scale
   - HPA at max replicas frequently → need higher max or bigger pods

4. **Cost-aware planning:**
    - Use Spot/preemptible for stateless workloads (60-70% savings)
    - Reserved instances for baseline, on-demand for burst
    - Right-size instances quarterly (AWS Compute Optimizer)
5. **Load testing:** Regular load tests at projected capacity before peak events.

Section 6: Cloud & Architecture (Q33–Q37)

Q33. How do you design a multi-account AWS strategy for platform engineering?

Answer:

```
Organization Root
├── Security OU
│   ├── Log Archive Account (CloudTrail, Config logs)
│   └── Security Tooling Account (GuardDuty, Security Hub)
├── Infrastructure OU
│   ├── Shared Services Account (CI/CD, Container Registry, DNS)
│   └── Networking Account (Transit Gateway, VPN, Direct Connect)
├── Workloads OU
│   ├── Dev Account
│   ├── Staging Account
│   └── Production Account
└── Sandbox OU
    └── Developer Sandbox Accounts
```

Why multi-account:

- **Blast radius isolation** — Compromise in dev can't reach prod
- **Billing separation** — Cost attribution per team/environment
- **IAM boundary** — Separate IAM policies per account
- **Service quotas** — Each account has independent limits

Q34. What is FinOps and how does SRE contribute?

Answer:

FinOps = Financial Operations. Making cloud cost a first-class engineering metric.

SRE contributions:

1. **Right-sizing:** Use Prometheus data to identify over-provisioned pods/nodes
2. **Spot instances:** Run stateless workloads on Spot (Karpenter makes this easy)
3. **Idle resource cleanup:** Automated Lambda to terminate unused dev environments after 6 PM
4. **Reserved capacity:** Commit to baseline capacity for predictable workloads
5. **Cost alerts:** CloudWatch alarm if daily spend exceeds threshold
6. **Chargeback dashboards:** Grafana dashboard showing cost per team/namespace

Key metric: Cost per transaction. Not just total spend, but cost efficiency as the platform scales.

Q35. How do you implement service mesh? When is it worth the complexity?

Answer:

Service mesh (Istio, Linkerd) adds a sidecar proxy to every pod that handles:

- **mTLS** — Automatic encryption between all services
- **Traffic management** — Canary, blue-green, circuit breakers, retries
- **Observability** — Automatic metrics, traces, logs without app code changes
- **Access control** — Service-to-service authorization policies

When worth it:

- ✅ 20+ microservices with complex inter-service communication
- ✅ Strict security requirements (mTLS everywhere)
- ✅ Need traffic shifting for canary deployments without app changes

When NOT worth it:

- ❌ <10 services (overhead outweighs benefits)
- ❌ Team doesn't have expertise to operate it
- ❌ Latency-critical workloads (sidecar adds 1-3ms per hop)

My preference: Start without mesh. Use K8s NetworkPolicies for security, app-level retries for resilience. Add Linkerd (lighter than Istio) only when the complexity is justified.

Q36. How do you handle secrets rotation without downtime?

Answer:

Pattern: Dual-read during rotation

1. **Secrets Manager** stores the secret with automatic rotation (Lambda-based)
2. Rotation Lambda:
    - Creates new credential (e.g., new DB password)
    - Updates secret in Secrets Manager (new version)
    - Verifies new credential works
    - Marks old credential for deletion (grace period)
3. **Application reads secret at startup** and caches it
4. **External Secrets Operator** syncs new secret to K8s Secret (1h refresh)
5. **Rolling restart** of pods to pick up new secret

Zero-downtime trick: Application checks both old and new credentials. If new fails, falls back to old. Or use connection poolers (PgBouncer, ProxySQL) that can hot-swap credentials.

Q37. Describe your ideal on-call setup.

Answer:

Structure:

- **Rotation:** Weekly, 2-person team (primary + secondary)

- **Hours:** Follow-the-sun if possible, otherwise compensated for off-hours
- **Handoff:** 30-min overlap. Outgoing on-call briefs incoming on active issues.
- **Escalation:** Primary doesn't respond in 5 min → Secondary paged. Secondary doesn't respond → Engineering Manager.

Tooling:

- **PagerDuty/OpsGenie** for alerting and scheduling
- **Runbooks** for every alert (linked in alert annotation)
- **Status page** (Statuspage.io) for customer communication
- **Slack incident channel** (auto-created per incident)

Compensation and sustainability:

- On-call pay or comp time
- Max 2 incidents per shift target (if more → too much toil)
- Monthly review: Are alerts actionable? Is on-call sustainable?
- **Error budget policy:** If error budget exhausted, feature freeze — not more on-call pressure

Post on-call: Write up any incidents, update runbooks, file toil-reduction tickets.

Quick Reference — Key Numbers

Metric	Value
99.9% SLO error budget	43.2 min/month
99.95% SLO error budget	21.6 min/month
99.99% SLO error budget	4.3 min/month
K8s pod max graceful shutdown	30s default (configurable)
EKS max pods per node (t3.medium)	17 (ENI-based)
Prometheus retention default	15 days
Lambda max timeout	15 min
S3 durability	99.999999999% (11 nines)
DynamoDB SLA	99.999% (Global Tables)

> **Tip:** For a 7-10 year SRE role, interviewers expect you to **lead discussions**, not just answer. Have opinions on trade-offs, share war stories, and demonstrate that you've built and operated real systems at scale.